

Towards Scalable Safety Verification of Neural Barrier Functions for Nonlinear Systems

Hongchao Zhang
Zezhang Yang
Shiyu Chen
Andrew Clark

ABSTRACT

Safety is a critical property of cyber-physical systems. An emerging approach to safe control of CPS is to train a neural barrier function (NBF) that approximates the safe region of the system. While several algorithms have been proposed for training NBFs, verifying that the safe region defined by the barrier function (i) is positive invariant and (ii) does not intersect with obstacles or other unsafe areas of the state space is computationally difficult. In this paper, we propose a framework for efficient safety verification of feedforward NBFs with ReLU activation functions. Our approach leverages two insights. First, it suffices to verify safety only at a relatively small subset of critical states at the boundary of the safe region, which we provide an algorithm to characterize. Second, since the ReLU function is piecewise linear, the safety verification problem can be decomposed into a set of subproblems, each of which can be solved using nonlinear programming. Moreover, when the system dynamics are polynomial, the verification subproblems can be solved via convex sum-of-squares optimization. Our approach is able to incorporate actuation constraints, and includes a novel equivalent condition for positive invariance of a level set of a neural network. We evaluated our approach by verifying trained neural barrier functions for two nonlinear systems. Our experiments show our approach is capable of verifying a 128-neuron NBF on a laptop computer.

CCS CONCEPTS

• **General and reference** → **Verification**; • **Computing methodologies** → *Neural networks*; • **Computer systems organization** → *Embedded and cyber-physical systems*.

KEYWORDS

Verification, Neural Networks, Control Barrier Function, Cyber-physical Systems

ACM Reference Format:

Hongchao Zhang, Zezhang Yang, Shiyu Chen, and Andrew Clark. 2025. Towards Scalable Safety Verification of Neural Barrier Functions for Nonlinear Systems. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA

© 2025 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . \$15.00
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Safety is a critical property of control systems including unmanned ground, aerial, and space vehicles [2] and robotic manipulators [16]. From a control-theoretic perspective, safety can be modeled as ensuring that the system state avoids an unsafe set, for example, avoiding obstacles or maintaining velocity or attitude within a desired tolerance region.

The importance of safety has motivated extensive research into verification and synthesis of safe control strategies for autonomous systems [31]. In particular, energy-based methodologies such as barrier certificates [23] and control barrier functions [5] have been proposed that construct an energy function that is positive if the system is safe, and then prove safety by proving that the system remains positive for all time. These methods generalize classic Lyapunov-methods for proving stability [6]. Control barrier functions provide a high degree of flexibility, since any control policy that satisfies the control barrier function constraint is provably safe.

A key challenge in energy-based safe control is constructing the energy function. An incorrect choice of energy function may fail to ensure safety or lead to a design that is too conservative. Typically, energy functions are chosen by hand using known physical laws, however, this approach may fail if such an energy function is not readily available. In [4], sum-of-squares algorithms have been proposed for automatically constructing barrier certificates with provable optimality guarantees. These approaches, however, are limited to polynomial functions, and typically do not scale to high-dimensional nonlinear systems.

More recently, *neural barrier functions* (NBFs) have been proposed to address the limitations of polynomial barrier functions [1, 9, 24]. NBFs are represented by neural networks, and instead of being chosen by hand or by solving an optimization problem, they are trained by providing examples of safe and unsafe states [19, 34]. The advantage of NBFs arises from the universal representability of neural networks, which allows them to realize safety constraints that cannot be encoded as low-degree polynomials.

In order to ensure trust and confidence in safety-critical autonomous CPS, the safety properties of NBF-based controllers must be verified. This verification is challenging, however, because it requires joint consideration of the neural network and the underlying nonlinear system dynamics. Since neural networks are not readily interpretable, human domain knowledge cannot be used to mitigate the challenge of verifying the system. Hence, existing verification techniques rely on Satisfiability Modulo Theory (SMT)-based approaches [17, 18] that either approximate the neural network or exhaustively search over the entire state space.

In this paper, we consider the problem of verifying safety of a nonlinear control system using a NBF represented by a feedforward

neural network with ReLU activation function. We choose the ReLU function because of its widespread use in NN learning including for NBFs. We reduce the complexity of the verification by exploiting two key features of the verification problem. First, it is sufficient to verify safety at a subset of critical states that lie at the boundary between the safe and unsafe regions. We propose an algorithm for characterizing the set of such critical states. Second, a ReLU-based NBF is piecewise linear, which enables us to decompose the verification problem into a collection of nonlinear programs. In particular, when the system dynamics are polynomial, each sub-problem can be solved using sum-of-squares optimization. We make the following specific contributions:

- We prove that safety verification of an NBF is equivalent to verifying safety at a subset of the boundary of the safe region. We characterize this boundary as the union of a collection of inputs that activate different subsets of neurons, and propose an algorithm for constructing the safety boundary by enumerating all such activation sets.
- We formulate a novel necessary and sufficient condition for positive invariance of the level set of a neural network. This condition generalizes the safety conditions of control barrier functions to barrier functions where the gradient of the barrier function is not well-defined.
- We formulate the problem of ensuring that, for each verification region, there is no system state that leads to safety violation. We map this problem to a nonlinear program and incorporate constraints on actuation. For polynomial systems, we prove equivalence to a sum-of-squares program via the algebraic geometric Positivstellensatz.
- We evaluate our approach through numerical study, in which our proposed approach achieves a five-fold reduction in verification time compared to state-of-the-art SMT-based methods [25].

The remainder of the paper is organized as follows. Section 2 reviews the related work. Section 3 gives the system model and background on neural networks and algebraic geometry. Section 4 presents the problem formulation and an overview of the solution approach. Section 5 presents our approach to verifying each sub-region of the safety boundary and gives the overall safety proof of our approach. Section 6 contains simulation results. Section 7 concludes the paper.

2 RELATED WORK

Safety of control systems has been extensively studied, mostly within the framework of ensuring positive invariance of a given safe region. See [10] and [7] for extensive treatments. Energy-based methods, in which safety is verified by ensuring that a particular energy function remains nonnegative, comprise an important class of safe control methodologies [28]. Barrier certificates for safe control were first proposed in [23]. More recently, control barrier functions have been developed for minimally-invasive safe controller design [5] and implemented in a variety of application domains [6, 16]. Traditionally, barrier functions for safety verification have either been hand-designed using domain knowledge or computed via sum-of-squares optimization, with the latter class of methods mainly used for low-dimensional applications with polynomial dynamics.

Neural barrier certificates, in which the barrier function is represented as a neural network, have recently emerged as a promising approach to improving scalability and generalizability of energy-based safe control. Among these methods, neural barrier certificates learn a barrier function that describes the safety of a given dynamical system [1], while neural control barrier functions jointly learn a control policy and a barrier function that certifies safety of that policy [14]. In both cases, the barrier function is typically learned by constructing a loss function that encodes the safety constraints of the system [11]. While most existing works focus on learning barrier certificates for systems with known dynamics (e.g., given as a nonlinear state-space model), some recent efforts have attempted to jointly learn a neural network model of the system dynamics, a control policy, and a barrier certificate [27, 29].

Since the barrier function is computed from training data which are inherently incomplete, the neural barrier function must be verified to ensure that it correctly enforces the safety constraint. Current methods for verification include Satisfiability Modulo Theory [25], output reachable set verification [30] and polynomial approximations of the barrier function [26]. These methods tend to scale exponentially in the problem dimension and neural network size; indeed, verification is often more time-consuming than the training process itself. Most existing neural barrier functions focus on systems with fewer than eight dimensions and neural networks with at most 40-50 neurons, due in part to the challenge of scalable verification [13]. Recently, in [35], an algorithm was proposed for synthesizing and verifying ReLU neural network controllers and barrier certificates by solving nonlinear mixed integer programs, and evaluated on neural networks consisting of hundreds of nodes.

The present paper differs from and builds on these related works in the following ways. First, rather than verifying safety of a particular neural network controller, our approach verifies whether *any* controller (including controllers that are not represented as neural networks) that is consistent with a particular neural CBF is safe. Hence, our approach is complementary to existing methods that train a neural network controller and/or barrier function. Second, our approach incorporates constraints on actuation. Third, we show that, instead of verifying the entire boundary of the safe region described by the barrier function, it suffices to verify a smaller subset of critical states. Finally, we derive novel necessary and sufficient conditions for a neural CBF to ensure safety that address the non-differentiability of the ReLU activation function.

A related body of work deals with the problem of verifying neural networks, e.g., determining whether a neural network output is non-negative for any input within a particular range. Significant recent advances, especially in verification of ReLU neural networks, have led to methodologies that scale to neural networks with hundreds or even thousands of neurons [17, 18]. These advances, however, are not directly applicable to the problem of verifying neural barrier certificates, since the latter problem requires joint consideration of the neural network and the nonlinear system dynamics.

Sum-of-squares optimization and other techniques derived from algebraic geometry have been widely used for safety verification of polynomial barrier functions [5, 23, 37]. These approaches, however, have seen only limited application in verification of neural networks [21, 26]. This paper describes how algebraic-geometric methods, specifically sum-of-squares optimization programs derived from

the Positivstellensatz [12, 22], can be leveraged to verify neural barrier functions, and hence aims to bridge traditional SOS-based verification with emerging neural barrier function methods.

3 MODEL AND PRELIMINARIES

In this section, we first present the system model. We then give preliminary background on the neural network model and sum-of-squares techniques.

3.1 System Model

Consider a continuous-time nonlinear control-affine system with state $x(t) \in \mathcal{X} \subseteq \mathbb{R}^n$, control input $u(t) \in \mathcal{U} \subseteq \mathbb{R}^m$, and dynamics

$$\dot{x}(t) = f(x(t)) + g(x(t))u(t), \quad (1)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $g : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ are known functions. We assume that the set $\mathcal{U}$ is polyhedral, i.e., there exist matrix $A \in \mathbb{R}^{p \times m}$ and vector $c \in \mathbb{R}^p$ such that $\mathcal{U} = \{u : Au \leq c\}$ where the inequality is element-wise.

We consider a collection of unsafe regions $U_1, \dots, U_M \subseteq \mathbb{R}^n$, where each U_i is defined by $U_i = \{x : h_i(x) \geq 0\}$ for some functions $h_1, \dots, h_M : \mathbb{R}^n \rightarrow \mathbb{R}$. We assume that each set U_i is connected.

We say that a trajectory $\{x(t) : t \geq 0\}$ is *safe* if for all $t \geq 0$, $x(t) \notin U_i$ for all $i = 1, \dots, M$, i.e., the trajectory never intersects an unsafe region. A safe control policy is defined as follows.

DEFINITION 1. A control policy $\mu : \mathcal{X} \rightarrow \mathcal{U}$ is safe from initial state x_0 if $x(0) = x_0$ and $u(t) = \mu(x(t))$ for all $t \geq 0$ implies that $x(t)$ is safe.

For any finite set V , we let $\mathcal{P}(V)$ denote the set of subsets of V (power set of V). For any set $C \subseteq \mathbb{R}^n$, we let ∂C denote the boundary of C , which is equal to the intersection of the closure of C with the closure of $\mathbb{R}^n \setminus C$. Given two sets S and T , we let

$$S \oplus T = S \setminus (S \cap T) \cup (T \setminus S).$$

3.2 Neural Network Model

We consider a feedforward neural network (NN) $b : \mathcal{X} \rightarrow \mathbb{R}$ with L layers and M_i neurons in the i -th layer. We let the input to the NN be denoted x , the state at the j -th neuron of the i -th layer be denoted z_{ij} , and the output of the network be denoted y . We let z_i denote the vector of neuron states at the i -th layer. The states and output are computed as

$$z_{ij} = \begin{cases} \sigma(W_{ij}^T x + r_{ij}), & i = 1 \\ \sigma(W_{ij}^T z_{i-1} + r_{ij}), & i \in \{2, \dots, L-1\} \end{cases}$$

$$y = \Omega^T z_L + \psi$$

where $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is the activation function. The input to the function σ is the *pre-activation input* to the neuron, and is given by $W_{1j}^T x + r_{1j}$ for the j -th neuron at the first layer and $W_{ij}^T z_{i-1} + r_{ij}$ for the j -th neuron at the i -th hidden layer. In this paper, we assume that σ is given by the ReLU function $\text{ReLU}(z) = \max\{0, z\}$. The output of the network is given by $y = \Omega^T z_L + \psi$, where $\Omega \in \mathbb{R}^{M_L}$ and $\psi \in \mathbb{R}$.

The j -th neuron at the i -th layer is *activated* by a particular input x if its pre-activation input is positive, *inactivated* if the pre-activation input is negative, and *unstable* if the pre-activation input is zero. An *activated set* $S = (S_1, \dots, S_L)$ denotes the set of neurons

$S_i \subseteq \{1, \dots, M_i\}$ that are activated at the i -th layer as well as a set of neurons that are unstable at that layer. For a given activation pattern S , the pre-activation input to each neuron and the overall output of the network are affine in x , with the affine mapping defined as follows. For the first layer, we define

$$\bar{W}_{1j}(S) = \begin{cases} W_{1j}, & j \in S_1 \\ 0, & \text{else} \end{cases}$$

$$\bar{r}_{1j}(S) = \begin{cases} r_{1j}, & j \in S_1 \\ 0, & \text{else} \end{cases}$$

so that the input to the j -th neuron at the first layer is $\bar{W}_{1j}(S)^T x + \bar{r}_{1j}(S)$. We recursively define $\bar{W}_{ij}(S)$ and $\bar{r}_{ij}(S)$ by letting $\bar{W}_i(S)$ be a matrix with columns $\bar{W}_{i1}(S), \dots, \bar{W}_{iM_i}(S)$ and

$$\bar{W}_{ij}(S) = \begin{cases} \bar{W}_{i-1}(S)W_{ij}, & j \in S_i \\ 0, & j \notin S_i \end{cases}$$

$$\bar{r}_{ij}(S) = \begin{cases} \bar{W}_{ij}(S)^T \bar{r}_{i-1}(S) + r_{ij}, & j \in S_i, \\ 0, & \text{else} \end{cases}$$

We define $\bar{W}(S) = \bar{W}_L(S)\Omega$ and $\bar{r}(S) = \Omega^T \bar{r}_L(S) + \psi$. Based on these notations, when an input x activates the set S , $z_{ij} = \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S)$ and $y = \bar{W}(S)^T x + \bar{r}(S)$. The set of inputs x that activate a particular set of neurons S is given by the polyhedron

$$\bar{\mathcal{X}}(S) \triangleq \bigcap_{i=1}^L \left(\bigcap_{j \in S_i} \{x : W_{ij}^T (\bar{W}_i(S)x + \bar{r}_i) + r_{ij} \geq 0\} \right. \\ \left. \cap \bigcap_{j \notin S_i} \{x : W_{ij}^T (\bar{W}_i(S)x + \bar{r}_i) + r_{ij} \leq 0\} \right). \quad (2)$$

We denote $\bar{\mathcal{X}}(S)$ as the activation region corresponding to activation set S . Note that in the definition of (2), a particular input x could belong to multiple $\bar{\mathcal{X}}(S)$ sets. We define the set of unstable neurons as T . We let $S(x) = \{S : x \in \bar{\mathcal{X}}(S)\}$ and let $T(x)$ denote the set of unstable neurons produced by input x . Given a collection of activation sets $S_1, \dots, S_r$, we let

$$T(S_1, \dots, S_r) = \left(\bigcup_{l=1}^r S_l \right) \setminus \left(\bigcap_{l=1}^r S_l \right).$$

The set $T(S_1, \dots, S_r)$ is equal to the set of neurons that must be unstable in order for an input x to belong to $\bar{\mathcal{X}}(S_1) \cap \dots \cap \bar{\mathcal{X}}(S_r)$.

We write that $S_1, \dots, S_r$ is *complete* if for any $T' \subseteq T(S_1, \dots, S_r)$, $(\bigcap_{l=1}^r S_l) \cup T' \in \{S_1, \dots, S_r\}$. We have the following preliminary result.

LEMMA 1. Let $S_1, \dots, S_r$ denote a collection of activation sets. Then $x \in \bigcap_{l=1}^r \bar{\mathcal{X}}(S_l)$ if and only if, for all $l = 1, \dots, r$,

$$\bar{W}_{ij}(S_l)^T x + \bar{r}_{ij}(S_l) \geq 0 \quad (3)$$

$$\forall (i, j) \in S_1 \cap \dots \cap S_r$$

$$\bar{W}_{ij}(S_l)^T x + \bar{r}_{ij}(S_l) \leq 0 \quad (4)$$

$$\forall (i, j) \notin S_1 \cup \dots \cup S_r$$

$$\bar{W}_{ij}(S_l)^T x + \bar{r}_{ij}(S_l) = 0 \quad (5)$$

$$\forall (i, j) \in T(S_1, \dots, S_r)$$

Moreover, we have

$$x \in \left(\bigcap_{l=1}^r \bar{X}(S_l) \right) \setminus \left(\bigcup_{S' \notin \{S_1, \dots, S_r\}} \bar{X}(S') \right) \quad (6)$$

if and only if $S_1, \dots, S_r$ is complete and (4) holds with strict inequality for all $l = 1, \dots, r$.

PROOF. Suppose that $x \in \bigcap_{l=1}^r \bar{X}(S_l)$. Eq. (3) and (4) follow by Eq. (2). If $(i, j) \in T(S_1, \dots, S_r)$, then there exist l, l' such that $(i, j) \in S_l$ and $(i, j) \notin S_{l'}$. In order for $x \in \bar{X}(S_l)$ to hold, the pre-activation input to neuron (i, j) must be nonnegative, whereas in order for $x \in \bar{X}(S_{l'})$ to hold, the pre-activation input to (i, j) must be nonpositive. Hence the pre-activation input to (i, j) must be zero, implying (5).

To prove the converse, suppose that (3)–(5) hold, and let $l \in \{1, \dots, r\}$. If $(i, j) \in S_l$, then (3) and (5) imply that $\bar{W}_{ij}(S_l)^T x + \bar{r}_{ij}(S_l) \geq 0$. If $(i, j) \notin S_l$, then (4) and (5) imply that $\bar{W}_{ij}(S_l)^T x + \bar{r}_{ij}(S_l) = 0$. Hence $x \in \bar{X}(S_l)$.

We next prove the equivalent condition for (6). Suppose that $x \in \bigcap_{l=1}^r \bar{X}(S_l)$ and for some $l = 1, \dots, r$, there exists $(i, j) \notin S_1 \cup \dots \cup S_r$ such that (4) holds with equality. Let $S' = S_l \cup \{(i, j)\}$. We then have that the pre-activation input to the nodes in S' is nonnegative, while the pre-activation inputs to the nodes not in S' is nonpositive, implying that $x \in \bar{X}(S')$ and contradicting (6).

Next, suppose that $S_1, \dots, S_r$ is not complete. Let $S' = (S_1 \cap \dots \cap S_r) \cup T'$ with $T' \subseteq T(S_1, \dots, S_r)$ such that $S' \neq \{S_1, \dots, S_r\}$. For all $(i, j) \in S'$, either $(i, j) \in S_1 \cap \dots \cap S_r$ and the pre-activation input to (i, j) is non-negative, or $(i, j) \in T(S_1, \dots, S_r)$ and the pre-activation input is zero and hence nonnegative. For all $(i, j) \notin S'$, either $(i, j) \in T(S_1, \dots, S_r)$ and the pre-activation input is zero, or $(i, j) \notin \bigcup_{l=1}^r S_l$ and the pre-activation input is nonpositive. Hence $x \in \bar{X}(S')$, contradicting (6).

Finally, suppose that (6) fails. Then there exists $S' \neq \{S_1, \dots, S_r\}$ such that $x \in (\bigcap_{l=1}^r S_l) \cap S'$. If $S' \subseteq \bigcup_{l=1}^r S_l$, then $S_1, \dots, S_r$ is not complete. If there is $(i, j) \notin S' \setminus (\bigcup_{l=1}^r S_l)$, then we must have that the pre-activation input to (i, j) is nonnegative, contradicting strict inequality of (4). $\square$

We let $\bar{X}(S, T)$ denote the set of inputs producing activation set S and unstable set T , which can be defined as

$$\bar{X}(S, T) = \bar{X}(S) \cap \bigcap_{(i,j) \in T} \{x : \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) = 0\}.$$

3.3 Algebraic Geometry and Sum-of-Squares Methods

A Sum-of-Squares (SOS) polynomial is a polynomial $F(x)$ such that $F(x) = \sum_{i=1}^k f_i(x)^2$ for some polynomials $f_1(x), \dots, f_k(x)$. An SOS program is an optimization problem to select coefficients of $f(x)$ ensuring that $f(x)$ is SOS. Such a program can be represented as a semi-definite program [3, 22].

The following result gives necessary and sufficient conditions for non-existence of solutions to systems of polynomial equations and inequalities. As a preliminary, for any set of polynomials $f_1, \dots, f_k$, we let $\Sigma[f_1, \dots, f_k]$ denote the cone defined by $f_1, \dots, f_k$, which is

given by

$$\Sigma[f_1, \dots, f_k] \triangleq \left\{ \alpha_0(x) + \sum_{R \subseteq \{1, \dots, k\}} \alpha_R(x) \prod_{i \in R} f_i(x) : \alpha_0, \alpha_R(x) \text{ SOS } \forall R \subseteq \{1, \dots, k\} \right\}.$$

We let $\mathcal{I}[f_1, \dots, f_k]$ denote the ideal generated by $f_1, \dots, f_k$, which is given by

$$\mathcal{I}[f_1, \dots, f_k] \triangleq \left\{ \sum_{i=1}^k \theta_i(x) f_i(x) : \theta_1(x), \dots, \theta_k(x) \text{ polynomials} \right\}.$$

Lastly, we let $\mathcal{M}[f_1, \dots, f_k]$ denote the multiplicative monoid defined by $f_1, \dots, f_k$, which is given by

$$\mathcal{M}[f_1, \dots, f_k] = \left\{ \prod_{i=1}^k f_i(x)^{n_i} : n_1, \dots, n_k \text{ nonnegative integers} \right\}.$$

THEOREM 1 (POSITIVSTELLENSATZ [22]). Let $(\phi_j)_{j=1, \dots, s^*} (\eta_k)_{k=1, \dots, t^*} (\psi_\ell)_{\ell=1, \dots, u}$ be finite families of polynomials in $\mathbb{R}[x_1, \dots, x_n]$. The following properties are equivalent:

(1) The set

$$\left\{ x \in \mathbb{R}^n \mid \begin{array}{ll} \phi_j(x) \geq 0, & j = 1, \dots, s \\ \eta_k(x) \neq 0, & k = 1, \dots, t \\ \psi_\ell(x) = 0, & \ell = 1, \dots, u \end{array} \right\}$$

is empty.

(2) There exist $\phi \in \Sigma[\phi_1, \dots, \phi_s]$, $\eta \in \mathcal{M}[\eta_1, \dots, \eta_t]$, and $\psi \in \mathcal{I}[\psi_1, \dots, \psi_u]$ such that $\phi + \eta^2 + \psi = 0$.

The next lemma gives sufficient conditions for solutions to a system of linear inequalities.

LEMMA 2 (FARKAS' LEMMA [20]). Let A be a real matrix with m rows and n columns, and let $b \in \mathbb{R}^m$ be a vector. Then the system of inequalities $Ax \leq b$ has a solution if and only if each y with $y \geq 0$ and $y^T A = 0^T$ also satisfies $y^T b \geq 0$, where 0 denote a zero vector.

4 PROBLEM FORMULATION

In this section, we formulate the problem of verifying a neural barrier function and present our solution approach.

4.1 Problem Statement

The problem studied in this paper is to verify a control barrier function that is encoded as a neural network $b : \mathcal{X} \rightarrow \mathbb{R}$ as described in Section 3.2. In the remainder of the paper, we let $C = \{x : b(x) \geq 0\}$.

ASSUMPTION 1. For all $i = 1, \dots, M$, we have that U_i is not contained in C .

Assumption 1 will be made for the remainder of the paper, and ensures that no unsafe region is contained entirely in C . We remark that this condition is straightforward to verify, since it suffices to find a single $x \in U_i$ and check if $b(x) \geq 0$; if $b(x) < 0$, then Assumption 1 holds.

Set C is controlled positive invariant if there exists a control policy $\mu : \mathcal{X} \rightarrow \mathcal{U}$ such that, if $x(0) \in C$ and $u(t) = \mu(x(t))$ for all time $t \geq 0$, then $x(t) \in C$ for all $t \geq 0$. In order to formally state

the problem, we first present the following preliminary definition and results.

DEFINITION 2. Let $\mathcal{A}$ be a closed set. The tangent cone to $\mathcal{A}$ at x is defined by

$$\mathcal{T}_{\mathcal{A}}(x) = \left\{ z : \liminf_{\tau \rightarrow 0} \frac{\text{dist}(x + \tau z, \mathcal{A})}{\tau} = 0 \right\} \quad (7)$$

We first describe the tangent cone for a special case of $\mathcal{A}$.

LEMMA 3 ([10]). Suppose that the set $\mathcal{A}$ is defined by

$$\mathcal{A} = \{x : q_k(x) \leq 0, k = 1, \dots, N\}$$

for some collection of differentiable functions $q_1, \dots, q_N$. For any x , let $J(x) = \{k : g_k(x) = 0\}$. Then

$$\mathcal{T}_{\mathcal{A}}(x) = \{z : z^T \nabla q_k(x) \leq 0 \forall k \in J(x)\}.$$

We have the following preliminary result.

THEOREM 2 (NAGUMO'S THEOREM [10], SECTION 4.2). A closed set $\mathcal{A}$ is controlled positive invariant if and only if, whenever $x(t) \in \partial\mathcal{A}$, $u(t) \in \mathcal{U}$ satisfies

$$(f(x(t)) + g(x(t))u(t)) \in \mathcal{T}_{\mathcal{A}}(x(t)) \quad (8)$$

A control policy that is consistent with a barrier function candidate is defined as follows.

DEFINITION 3. A control policy $\mu : \mathcal{X} \rightarrow \mathcal{U}$ is consistent with a barrier function $b : \mathbb{R}^n \rightarrow \mathbb{R}$ if for any x satisfying $b(x) = 0$ such that there exists $u \in \mathcal{U}$ satisfying (8) with $C = \{x : b(x) \geq 0\}$, we have $(f(x) + g(x)\mu(x)) \in \mathcal{T}_C(x)$.

Intuitively, a control policy is consistent with a barrier function candidate if the policy attempts to choose a control input satisfying positive invariance of $\{x : b(x) \geq 0\}$. A widely-studied example consists of optimization-based controllers where u is chosen at each time t by solving an optimization problem of the form

$$\begin{aligned} & \text{minimize} \quad \|u - u_{\text{nom}}(t)\| \\ & \text{s.t.} \quad \frac{\partial b}{\partial x}(f(x(t)) + g(x(t))u) \geq -\alpha(b(x(t))) \\ & \quad u \in \mathcal{U} \end{aligned}$$

where $u_{\text{nom}}(t)$ is the output of a nominal controller and α is a class-K function [5]; a similar policy for neural network barrier functions was defined in [36]. We are now ready to state the problem studied in the paper.

PROBLEM 1. Given a nonlinear continuous-time system (1), a neural network function $b : \mathcal{X} \rightarrow \mathbb{R}$, a set of admissible control inputs $\mathcal{U}$, an initial state $x_0 \in C$, and unsafe sets $U_1, \dots, U_M$ where $U_i = \{x : h_i(x) \geq 0\}$, verify whether any control policy that is consistent with b is safe from initial state x_0 .

The goal of Problem 1 is to verify that any control policy μ that is based on a CBF b is safe. Intuitively, a CBF-based control policy may fail to guarantee safety for two reasons. First, there may exist x on the boundary of C such that there is no $u \in \mathcal{U}$ satisfying (8), causing the system to leave the safe region regardless of the control input that is provided. Second, there may be overlap between C and one of the unsafe regions, leading to safety violations even if $b(x(t)) \geq 0$ is satisfied for all time t . In what follows, we describe our approach to determining whether these conditions are met.

4.2 Overview of Approach

We next present an overview of our approach to safety verification. We first identify a subset of the safe region that is critical to safety verification. We then characterize the tangent cone to each point in this subset of the safe region. Finally, we formulate a sufficient condition for safety. For any $x_0 \in C$, we let C_0 denote the connected component of C containing x_0 . We let Φ_i denote the connected component of $\mathbb{R}^n \setminus C_0$ containing $U_i \setminus C_0$ and $\Psi_i = \mathbb{R}^n \setminus \Phi_i$. Finally, let D_i denote the boundary of Ψ_i .

LEMMA 4. Given an initial state $x_0 \in C$, for each U_i , any continuous path $\gamma : [0, \infty) \rightarrow \mathbb{R}^n$ with $\gamma(0) = x_0$ and $\gamma(t') \in U_i$ for some $t' \geq 0$ satisfies at least one of the following: (i) $\gamma(t'') \in U_i \cap C$ for some $t'' \geq 0$, or (ii) $\gamma(t''') \in D_i$ for some $t''' \geq 0$. Furthermore, D_i is a connected component of the boundary of C_0 .

A proof can be found in the appendix. The following lemma characterizes the tangent cone to Ψ_i .

PROPOSITION 1. For any $x \in D_i$, we have

$$\mathcal{T}_{\Psi_i}(x) = \bigcup_{S \in \mathcal{S}(x)} \left(\bigcap_{(i,j) \in T(x)} \{z : \bar{W}_{ij}(S)^T z \geq 0\} \cap \{z : \bar{W}(S)^T z \geq 0\} \right) \quad (9)$$

A proof can be found in the appendix. The following result is a consequence of Proposition 1.

LEMMA 5. The set C is positive invariant if and only if, for all $x \in \partial C$, there exist $S \in \mathcal{S}(x)$ and $u \in \mathcal{U}$ satisfying

$$\begin{aligned} \bar{W}_{ij}(S)^T (f(x) + g(x)u) &\geq 0 \forall (i, j) \in T(x) \\ \bar{W}(S)^T (f(x) + g(x)u) &\geq 0 \end{aligned}$$

The proof follows immediately from Theorem 2 and the proof of Proposition 1 and is omitted due to space constraints. Based on Lemma 4 and Proposition 1, we now give an equivalent condition for avoiding each unsafe region.

PROPOSITION 2. The trajectory $x(t)$ satisfies $x(t) \notin U_i$ for all t for any control policy consistent with b if the following two properties hold:

- (i) For all activation sets $S_1, \dots, S_r$ with $\{S_1, \dots, S_r\}$ complete, we have

$$\bar{X}(S_1) \cap \dots \cap \bar{X}(S_r) \cap D_i \neq \emptyset \quad (10)$$

and any x satisfying $b(x) = 0$ and

$$x \in \left(\bigcap_{l=1}^r \bar{X}(S_l) \right) \setminus \left(\bigcup_{S' \notin \{S_1, \dots, S_r\}} \bar{X}(S') \right), \quad (11)$$

there exist $l \in \{1, \dots, r\}$ and $u \in \mathcal{U}$ such that

$$\bar{W}_{ij}(S_l)^T (f(x) + g(x)u) \geq 0 \forall (i, j) \in T(S_1, \dots, S_r) \quad (12)$$

$$\bar{W}(S_l)^T (f(x) + g(x)u) \geq 0 \quad (13)$$

- (ii) For all activation sets S , we have

$$\bar{X}(S) \cap D_i \cap U_i = \emptyset \quad (14)$$

PROOF. We first show that, if condition (ii) holds, then $U_i \cap C_0 = \emptyset$, and hence $U_i \subseteq \Phi_i$. We then show that (i) implies that Ψ_i is positive invariant, and hence the trajectory never reaches Φ_i .

Suppose that $x_1 \in U_i \cap C_0$. By Assumption 1, there exists $x_2 \in U_i \setminus C_0 \subseteq \Psi_i$. Since U_i is connected, there is a path from $x_1 \notin \Psi_i$ to $x_2 \in \Psi_i$ that is contained in U_i . Hence this path must cross the boundary of Ψ_i , which is equal to D_i , and we have $D_i \cap U_i \neq \emptyset$. Equivalently, if (ii) holds, then $U_i \cap C_0 = \emptyset$.

Next, suppose that condition (i) holds. Then for any $x \in D_i$ with $S(x) = \{S_1, \dots, S_r\}$, there exists $l \in \{1, \dots, r\}$ such that $x \in \bar{X}(S_l)$ and $u \in \mathcal{U}$ satisfying (12) and (13). For this choice of u , we have $(f(x) + g(x)u) \in \mathcal{T}_{\Psi_i}(x)$ by Proposition 1. Hence Ψ_i is positive invariant under any control policy consistent with b by Theorem 2, and hence $x(t) \notin U_i$ for all t when $x(0) = x_0$. $\square$

We refer to any x that fails to meet at least one of conditions (i) and (ii) of Proposition 2 as a safety counterexample.

The preceding proposition motivates our overall approach, which is summarized in the following steps:

- (1) For each i , draw a line from x_0 to a single point in U_i . Identify all points $x_{i1}, \dots, x_{iN_i}$ on the line that satisfy $b(x_{ij}) = 0$.
- (2) For each x_{ij} , let D_{ij} denote the connected component of the boundary of C containing x_{ij} . Compute the set S_{ij} of activation patterns such that $D_{ij} \subseteq \bigcup_{S \in S_{ij}} \bar{X}(S)$. For each $S \in S_{ij}$, verify that (14) holds.
- (3) For each i, j and $\{S_1, \dots, S_r\} \subseteq S_{ij}$ with $\bar{X}(S_1) \cap \dots \cap \bar{X}(S_r) \neq \emptyset$, verify that the conditions (12) and (13) are satisfied.

Our approach first finds points on the boundary between C and the unsafe region via line search (Step 1). It then reconstructs the critical connected components of the boundary as a union of activation sets (Step 2). Finally, for each activation set, we show that there is no safety counterexample corresponding to that activation set. A pseudocode description of this approach is given as Algorithm 1.

5 DETAILS OF APPROACH

In what follows, we describe each step of Algorithm 1. We first present an approach and algorithm for enumerating the activation sets at the boundary. We then map the problem of verifying safety of each activation set, as well as safety verification of intersections of activation sets, to nonlinear programs. Finally, we give a proof of correctness for our overall approach.

5.1 Enumerating Activation Sets

Our approach to enumerating the activation sets that intersect with the safety boundary is as follows. Starting with an initial point x with $b(x) = 0$ (which we compute by searching along a line between x_0 and U_i), we find an activation set $S \in \mathcal{S}(x)$. We then identify all sets S' such that there exists $x \in \bar{X}(S) \cap \bar{X}(S')$ with $b(x) = 0$; we observe that such an x can be computed by solving a linear program. We then find all sets S'' with $\bar{X}(S') \cap \bar{X}(S'') \cap \{x : b(x) = 0\} \neq \emptyset$, continuing until we have enumerated all such activation sets that are connected to x . This procedure is described in detail in Algorithms 2 and 3.

The following result describes the completeness guarantees of Algorithm 2.

PROPOSITION 3. *Let $\mathcal{S}$ and $\mathcal{V}$ denote the output of Algorithm 2. Then the connected component of $\{x : b(x) = 0\}$ containing the input x , denoted D , satisfies $D \subseteq \bigcup_{S \in \mathcal{S}} \bar{X}(S)$. Furthermore, if*

Algorithm 1 Verification of NCBF for Nonlinear Systems

```

1: Input: NCBF  $b(x)$ , Model  $f(x)$ ,  $g(x)$ , unsafe regions
    $U_1, \dots, U_M$ , initial state  $x_0$ 
2: Output: SAT or UNSAT
3: procedure VERIFY( $b(x)$ ,  $f(x)$ ,  $g(x)$ ,  $U_1, \dots, U_M$ ,  $x_0$ )
4:   for  $i = 1, \dots, M$  do
5:      $y_i \leftarrow$  point chosen in  $U_i$ 
6:      $\gamma_i \leftarrow$  line connecting  $x_0$  and  $y_i$ 
7:      $\{x_1, \dots, x_{iN_i}\} \leftarrow \{x : b(x) = 0\} \cap \gamma_i$ 
8:      $(S_{ij}, \mathcal{V}_{ij}) \leftarrow$  EnumerateSets( $x_{ij}$ )
9:     for  $S \in S_{ij}$  do
10:      if  $\max \{h_i(x) : x \in \mathcal{X}(S), b(x) = 0\} \geq 0$  then
11:        return UNSAT
12:      end if
13:      if minimum value of (15) is negative then
14:        return UNSAT
15:      end if
16:    end for
17:    for  $\{S_1, \dots, S_r\} \in \mathcal{V}_{ij}$  do
18:      if minimum value of (20) is negative then
19:        return UNSAT
20:      end if
21:    end for
22:  end for
23:  return SAT
24: end procedure

```

Algorithm 2 Enumerate Activated Sets

```

1: Input: State  $x$  such that  $b(x) = 0$ 
2: Output: Set of activation sets  $\mathcal{S}$  such that the connected compo-
   nent of the boundary containing  $x$  is contained in  $\bigcup_{S \in \mathcal{S}} \bar{X}(S)$ ,
   set of activation sets with nonempty intersection  $\mathcal{V}$ 
3: procedure ENUMERATESETS( $x$ )
4:   Compute  $S \in \mathcal{S}(x)$ 
5:   Return ActivationSetHelper( $S$ ,  $\{S\}$ ,  $\{S\}$ )
6: end procedure

```

$S_1, \dots, S_r \in \mathcal{S}$ is complete and $\left(\bigcap_{i=1}^r \bar{X}(S_i)\right) \cap \{x : b(x) = 0\} \neq \emptyset$, then $\{S_1, \dots, S_r\} \in \mathcal{V}$.

PROOF. Suppose that $x' \in D \setminus \left(\bigcup_{S \in \mathcal{S}} \bar{X}(S)\right)$. Since D is connected, there exists a path γ with $\gamma(0) = x$ and $\gamma(1) = x'$ contained in D . Let $S_0, S_1, \dots, S_K$ denote a sequence of activation patterns with $S_0 \in \mathcal{S}(x)$, $S_K \in \mathcal{S}(x')$, and S_0 equal to the set computed at Line 3 of Algorithm 2. Then there exists $i \geq 2$ such that $S_{i-1} \in \mathcal{S}$ and $S_i \notin \mathcal{S}$, and there exists t' such that $\gamma(t') \in \bar{X}(S_{i-1}) \cap \bar{X}(S_i) \cap \{x : b(x) = 0\}$. We then have that $T(S_{i-1}, S_i)$ is a subset of the set T computed at Line 4 of Algorithm 3, and hence S_i will be identified and added to $\mathcal{S}$ at Line 7 of Algorithm 3.

Now, suppose that $S_1, \dots, S_r \in \mathcal{S}$ is complete and $\left(\bigcap_{i=1}^r \bar{X}(S_i)\right) \cap \{x : b(x) = 0\} \neq \emptyset$. Let l denote the first index such that Algorithm 3 is called with S_l as input; without loss of generality, suppose $l = 1$. Let $T = T(S_1, \dots, S_r)$. Then T is a subset of the set T

Algorithm 3 Helper function for recursively enumerating activation sets

```

1: Input: Initial set  $S$ , collection of activation sets that have already been enumerated  $\mathcal{S}$ , collection of intersections that have been identified  $\mathcal{V}$ 
2: Output: Updated lists of activated sets
3: procedure ACTIVATIONSETHelper( $S, \mathcal{S}, \mathcal{V}$ )
4:    $S' \leftarrow \emptyset$ 
5:    $T \leftarrow \{(i, j) : \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) = 0 \text{ for some } x \in \bar{X}_0(S)\}$ 
6:   for  $T' \subseteq T, T' \neq \emptyset$  do
7:      $r \leftarrow 2^{|T'|}$ 
8:      $\{T'_1, \dots, T'_r\} \leftarrow \mathcal{P}(T')$ 
9:     for  $l = 1, \dots, r$  do
10:       $S_l \leftarrow S \oplus T_l$ 
11:      if  $S_l \notin \mathcal{S} \cup S'$  then
12:         $S' \leftarrow S' \cup \{S_l\}$ 
13:      end if
14:    end for
15:    if  $\{S_1, \dots, S_r\} \notin \mathcal{V}$  then
16:      if  $\exists x \in \bar{X}(S_1) \cap \dots \cap \bar{X}(S_r) \cap \{b(x) = 0\}$  then
17:         $\mathcal{V} \leftarrow \mathcal{V} \cup \{\{S, S_1, \dots, S_r\}\}$ 
18:      end if
19:    end if
20:  end for
21:   $S \leftarrow S \cup S'$ 
22:  for  $S' \in S'$  do
23:     $(S, \mathcal{V}) \leftarrow \text{ActivationSetHelper}(S', S, \mathcal{V})$ 
24:  end for
25:  Return  $(S, \mathcal{V})$ 
26: end procedure

```

computed at line 4 of the algorithm, and the sets $S_1, \dots, S_r$ are enumerated at lines 7-13. Since the intersection of the $\bar{X}(S)$ sets with $\{x : b(x) = 0\}$ is nonempty, $\{S_1, \dots, S_r\}$ is added to $\mathcal{V}$. $\square$

5.2 Verification of a Single Activation Set

The following result gives an equivalent condition for verifying that there is no safety counterexample in the interior of $\bar{X}(S)$ for a given activation set S .

LEMMA 6. *There is no safety counterexample in the set $\bar{X}(S) \setminus \bigcup_{S' \neq S} \bar{X}(S')$ if and only if:*

- (1) *There do not exist $x \in \mathcal{X}$ and $y \in \mathbb{R}^{p+1}$ satisfying (a) $x \in \text{int}(\bar{X}(S))$, (b) $b(x) = 0$, (c) $y \geq 0$, (d) $y^T \begin{pmatrix} -\bar{W}(S)^T g(x) \\ A \end{pmatrix} = 0$, and (e) $y^T \begin{pmatrix} \bar{W}(S)^T f(x) \\ c \end{pmatrix} < 0$.*
- (2) *For all $i = 1, \dots, M$, there does not exist $x \in \mathcal{X}$ with $b(x) = 0$, $x \in \text{int}(\bar{X}(S))$, and $h_i(x) \geq 0$.*

PROOF. The proof is to show that conditions (1a)–(1e) are equivalent to the condition (i) in Proposition 2, while condition 2 is equivalent to condition (ii) of Proposition 2. The proof of the latter statement is clear. To show equivalence of (1a)–(1e) with condition

(i) of Proposition 2, we first observe that Eq. (11) is equivalent to $x \in \text{int}(\bar{X}(S))$. Since $r = 1$, we have that Eq. (12) is vacuous.

It remains to verify that there does not exist x such that, for any $u \in \mathcal{U}$, (13) fails to hold. This is equivalent to nonexistence of solutions to the system of linear inequalities

$$-\bar{W}(S)^T g(x)u \leq \bar{W}(S)^T f(x), \quad Au \leq c.$$

By Farkas's Lemma, this is equivalent to the existence of y satisfying (1c)–(1e), completing the proof. $\square$

Condition 1 of Lemma 6 can be verified by solving the nonlinear program

$$\begin{aligned}
 \min_{x,y} \quad & y^T \begin{pmatrix} \bar{W}(S)^T f(x) \\ c \end{pmatrix} \\
 \text{s.t.} \quad & y^T \begin{pmatrix} -\bar{W}(S)^T g(x) \\ A \end{pmatrix} = 0 \\
 & y \geq 0 \\
 & \bar{W}(S)^T x + \bar{r}(S) = 0 \\
 & \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) > 0 \quad \forall (i, j) \in S \\
 & \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) < 0 \quad \forall (i, j) \notin S
 \end{aligned} \tag{15}$$

and checking whether the optimal value is nonnegative (safe) or negative (unsafe).

The following corollary describes the special case where there are no constraints on the control, i.e., $\mathcal{U} = \mathbb{R}^m$.

COROLLARY 1. *If $\mathcal{U} = \mathbb{R}^m$, then there is no safety counterexample in the set $\bar{X}(S) \setminus \bigcup_{S' \neq S} \bar{X}(S')$ if and only if (1) there does not exist $x \in \mathcal{X}$ satisfying $x \in \text{int}(\bar{X}(S))$, $b(x) = 0$, $\bar{W}(S)^T g(x) = 0$, and $\bar{W}(S)^T f(x) < 0$ and (2) for all $i = 1, \dots, M$, there does not exist $x \in \mathcal{X}$ with $b(x) = 0$, $x \in \text{int}(\bar{X}(S))$, and $h_i(x) \geq 0$.*

Under the conditions of Corollary 1, the nonlinear program (15) reduces to

$$\begin{aligned}
 \min_x \quad & \bar{W}(S)^T f(x) \\
 \text{s.t.} \quad & \bar{W}(S)^T g(x) = 0 \\
 & \bar{W}(S)^T x + \bar{r}(S) = 0 \\
 & \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) > 0 \quad \forall (i, j) \in S \\
 & \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) < 0 \quad \forall (i, j) \notin S
 \end{aligned} \tag{16}$$

If $g(x)$ is a constant matrix G , then safety is automatically guaranteed if $\bar{W}(S)^T G \neq 0$. If $f(x)$ is linear in x as well, then (15) is a linear program.

Turning to Condition 2, we have that it suffices to check that for every connected component U_i of the unsafe region, $U_i \cap \{x : b(x) = 0\}$ is empty. Hence it suffices to solve the nonlinear program

$$\begin{aligned}
 \text{maximize} \quad & h(x) \\
 \text{s.t.} \quad & \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) \geq 0 \quad \forall (i, j) \in S \\
 & \bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) \leq 0 \quad \forall (i, j) \notin S \\
 & \bar{W}(S)^T x + \bar{r}(S) = 0
 \end{aligned} \tag{17}$$

and check whether the optimal value is nonnegative (unsafe) or negative (safe).

We next give a necessary and sufficient SOS condition for a given activation set to be safe using the Positivstellensatz when f , g , and h_i are polynomials. As a preliminary, define matrix $A(S)$ and $c(S) \in \mathbb{R}^n$ such that $\bar{X}(S) = \{x : A(S)x \leq c(S)\}$.

THEOREM 3. *There is no safety counterexample in the set $\bar{X}(S) \setminus \bigcup_{S' \neq S} \bar{X}(S)$ if and only if:*

- (1) *There exist $\eta \in \mathcal{I} \left[b, y^T \begin{pmatrix} -\bar{W}(S)^T g(x) \\ A \end{pmatrix} \right]$,*
 $\lambda \in \Sigma \left[A(S)x - c(S), y_1, \dots, y_{p+1}, -y^T \begin{pmatrix} \bar{W}(S)^T f(x) \\ c \end{pmatrix} \right]$,
and $\theta \in \mathcal{M} \left[A(S)x - c(S), y^T \begin{pmatrix} \bar{W}(S)^T f(x) \\ c \end{pmatrix} \right]$ satisfying
 $\lambda + \eta + \theta^2 = 0$.
- (2) *For all $i = 1, \dots, M$, there exist $\eta_i \in \mathcal{I} [b]$,*
 $\lambda_i \in \Sigma [A(S)x - c(S), h_i(x)]$, *and $\theta_i \in \mathcal{M} [A(S)x - c(S)]$*
such that $\lambda_i + \eta_i + \theta_i^2 = 0$.

PROOF. The approach is to show that conditions (1) and (2) of Theorem 3 are equivalent to conditions (1) and (2) of Lemma 6, respectively. This equivalence follows directly from the Positivstellensatz (Theorem 1). $\square$

Theorem 3 gives necessary and sufficient convex conditions for safety of the interior of an activation set.

5.3 Verification of Activation Set Boundaries

We now describe an approach for verifying safety of intersections between activation sets.

LEMMA 7. *Suppose that the sets $S_1, \dots, S_r$ satisfy condition (ii) of Lemma 6. There is no safety counterexample in the set*

$$\bar{X}(S_1) \cap \dots \cap \bar{X}(S_r) \setminus \bigcup_{S' \notin \{S_1, \dots, S_r\}} \bar{X}(S')$$

if and only if there do not exist $x \in \mathcal{X}$ and $y_1, \dots, y_r \in \mathbb{R}^{T+p+1}$, where $T = |\mathbf{T}(S_1, \dots, S_r)|$, satisfying (a) $x \in \text{int}(\bar{X}(S_1)) \cap \dots \cap \text{int}(\bar{X}(S_r))$, (b) $b(x) = 0$, (c) $y_l \geq 0$ for all $l = 1, \dots, r$, (d) for all $l = 1, \dots, r$

$$y_l^T \underbrace{\begin{pmatrix} -W_{i_1 j_1}(S_l)^T g(x) \\ \vdots \\ -W_{i_T j_T}(S_l)^T g(x) \\ -W(S_l)^T g(x) \\ A \end{pmatrix}}_{\triangleq \Theta_l(S_1, \dots, S_r, x)} = 0, \quad (18)$$

and (e) for all $l = 1, \dots, r$,

$$y_l^T \underbrace{\begin{pmatrix} \bar{W}_{i_1 j_1}(S_l)^T f(x) \\ \vdots \\ \bar{W}_{i_T j_T}(S_l)^T f(x) \\ \bar{W}(S_l)^T f(x) \\ c \end{pmatrix}}_{\triangleq \Lambda_l(S_1, \dots, S_r, x)} < 0 \quad (19)$$

PROOF. The approach is to prove that condition (ii) of Proposition 2 holds; condition (i) holds automatically if each $S_1, \dots, S_r$ satisfies condition (ii) of Lemma 6. We have that conditions (a) and (b) are equivalent to $b(x) = 0$ and (11). In order for x to be a safety counterexample, for all $l = 1, \dots, r$, at least one of Eqs. (12) and (13)

must fail. Equivalently, for all $l = 1, \dots, r$, there does not exist u satisfying

$$\begin{aligned} -\bar{W}_{ij}(S_l)^T g(x)u &\leq \bar{W}_{ij}(S_l)^T f(x) \quad \forall (i, j) \in T(S_1, \dots, S_r) \\ -\bar{W}_{ij}(S_l)^T g(x)u &\leq \bar{W}(S_l)^T f(x) \\ Au &\leq c \end{aligned}$$

Non-existence of such a u is equivalent to existence of y_l satisfying $y_l \geq 0$ as well as (18) and (19). $\square$

The verification problem can then be mapped to solving the nonlinear program

$$\begin{aligned} \min_{x, y_1, \dots, y_r} \quad & \max_{l=1, \dots, r} \left\{ y_l^T \begin{pmatrix} \bar{W}_{i_1 j_1}(S_l)^T f(x) \\ \vdots \\ \bar{W}_{i_T j_T}(S_l)^T f(x) \\ c \end{pmatrix} \right\} \\ \text{s.t.} \quad & \bar{W}_{ij}(S_l)^T x + \bar{r}_{ij}(S_l) < 0 \quad \forall (i, j) \notin S_1 \cup \dots \cup S_r \\ & \bar{W}_{ij}(S_l)^T x + \bar{r}_{ij}(S_l) > 0 \quad \forall (i, j) \in S_1 \cap \dots \cap S_r \\ & \bar{W}_{ij}(S_l)^T x + \bar{r}_{ij}(S_l) = 0 \quad \forall (i, j) \in \mathbf{T}(S_1, \dots, S_r) \\ & \text{Eq. (18)} \quad \forall l = 1, \dots, r \\ & y_l \geq 0 \quad \forall l = 1, \dots, r \end{aligned} \quad (20)$$

and checking whether the optimal value is nonnegative (safe) or negative (unsafe). Note that, if $g(x)$ is constant, then the constraints of (20) are linear, and if $f(x)$ is linear then (20) reduces to solving a system of linear and bilinear inequalities. Furthermore, if $f(x)$ can be bounded over $x \in \bar{X}(S_1) \cap \dots \cap \bar{X}(S_r)$ and $b(x) = 0$, then this bound can be used to derive a linear relaxation to the objective function of (20), thus relaxing the nonlinear program to a linear program.

Next, we derive convex conditions for safety verification of the intersection of multiple activation regions for systems with polynomial dynamics via the Positivstellensatz.

THEOREM 4. *There is no safety counterexample in the set*

$$\bar{X}(S_1) \cap \dots \cap \bar{X}(S_r) \setminus \bigcup_{S' \notin \{S_1, \dots, S_r\}} \bar{X}(S')$$

if and only if there exist

$$\begin{aligned} \lambda \in \Sigma [A(S_1) - c(S_1), \dots, A(S_r) - c(S_r), y_1, \dots, y_r, \\ (-y_l^T \Theta_l(S_1, \dots, S_r, x) : l = 1, \dots, r)], \end{aligned}$$

$$\eta \in \mathcal{I} [b, (\bar{W}_{ij}(S_1)^T x + \bar{r}_{ij}(S_1)), (y_l^T \Lambda_l(S_1, \dots, S_r, x))],$$

and

$$\begin{aligned} \theta \in \mathcal{M} [A(S_1) - c(S_1), \dots, A(S_r) - c(S_r), \\ (y_l^T \Theta_l(S_1, \dots, S_r, x) : l = 1, \dots, r)] \end{aligned}$$

such that $\lambda + \eta + \theta^2 = 0$.

PROOF. Follows by applying Positivstellensatz (Theorem 1) to the conditions of Lemma 7. $\square$

5.4 Proof of Correctness

The following result summarizes the correctness of our overall approach.

THEOREM 5. *For each $i = 1, \dots, M$, let $\mathcal{Z}_i$ denote the set of points where a line from x_0 to U_i intersects $\partial\mathcal{C}$. For each $x_{ij} \in \mathcal{Z}_i$, let $(S_{ij}, \mathcal{V}_{ij})$ denote the output of Algorithm 2 when x_{ij} is given as input. Suppose that the following conditions hold: (a) For all $S \in S_{ij}$, the value of the nonlinear program (15) is nonnegative; (b) For all $S \in S_{ij}$, the value of the nonlinear program $\min \{h(x) : x \in \bar{\mathcal{X}}_0(S)\}$ is negative; and (c) For all $\{S_1, \dots, S_r\} \in \mathcal{V}_{ij}$, the value of the nonlinear program (20) is nonnegative. Then any control policy that is consistent with b is safe.*

PROOF. The approach is to show that the conditions of Proposition 2 are satisfied. By Lemma 4, for all i , there exists $x_{ij} \in \mathcal{Z}_{ij}$ such that the connected component of the boundary containing x_{ij} is equal to D_i . We have that, by Proposition 3, any set $S_1, \dots, S_r$ satisfying (10) is contained in $\mathcal{V}_{ij}$. If $r = 1$, then condition (a) and Lemma 6 imply that (12) and (13) hold for all x in $\bar{\mathcal{X}}_0(S_1)$. If $r > 1$, then condition (c) and Lemma 7 imply that (12) and (13) hold for all x at the intersection of $\bar{\mathcal{X}}(S_1), \dots, \bar{\mathcal{X}}(S_r) \cap \{x : b(x) = 0\}$.

Finally, by Proposition 3, the set of S with $\bar{\mathcal{X}}(S) \cap D_i \neq \emptyset$ is contained in S_{ij} . By condition (b), these sets have no intersection with U_i , implying that (14) is satisfied. $\square$

Theorem 5 implies that, if Algorithm 1 returns SAT, then any control policy consistent with barrier function b will be safe.

6 CASE STUDIES

In this section, we evaluate our proposed method to verify neural barrier functions on two nonlinear systems. We first consider the Darboux system [32], a nonlinear open-loop polynomial system that has been widely used for safety verification. We then evaluate our approach on an obstacle avoidance scenario [8].

We trained the neural barrier functions to be verified via the approach proposed in [33], which we summarize as follows. We first sample points from the state space, initial region, and unsafe region separately to generate data for training denoted as D_D , D_I , and D_U respectively. We then trained our NBFs and NCBFs with loss function

$$C(D_D, D_I, D_U) = \sum_{x \in D_D} C_D(x) + \sum_{x \in D_I} C_I(x) + \sum_{x \in D_U} C_U(x),$$

where $C_D, C_I, C_U : \mathbb{R}^n \rightarrow \mathbb{R}$ are sub-loss functions derived from Theorem 2.3 of [33].

We verified the neural barrier functions using the approach of Algorithm 1. We denote the time required to enumerate the activated sets (Algorithm 2) as t_e and the time required to verify that the corresponding activation regions do not intersect with U_i as t_b . We let $n(S)$ and $n(\mathcal{V})$ denote the number of activated sets and non-empty intersections of activated sets at the boundary $\partial\mathcal{C}$, respectively. We denote the time required to verify that there are no safety counterexamples in the interior of each activation region (as described in Section 5.2) as t_S and the time required to verify that there are no safety counterexamples at the boundaries of the activation regions (as described in Section 5.3) as t_V . The verification program was coded in Python and MATLAB and run

on a 64-bit Windows laptop with AMD Ryzen 7 5800H processor, 32GB RAM and NVIDIA GeForce RTX 3050 Ti Laptop GPU.

6.1 Darboux

The Darboux system has dynamics

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 + 2x_1x_2 \\ -x_1 + 2x_1^2 - x_2^2 \end{bmatrix}. \quad (21)$$

We denote state space, initial region, and unsafe region as $\mathcal{X}$, $\mathcal{X}_I$ and $\mathcal{X}_U$, respectively, and define them as

$$\begin{aligned} \mathcal{X} &: \{x \in \mathbb{R}^2 : x \in [-2, 2] \times [-2, 2]\} \\ \mathcal{X}_I &: \{x \in \mathbb{R}^2 : 0 \leq x_1 \leq 1, 1 \leq x_2 \leq 2\} \\ \mathcal{X}_U &: \{x \in \mathbb{R}^2 : x_1 + x_2^2 \leq 0\} \end{aligned} \quad (22)$$

We construct a neural barrier function with one hidden layer consisting of 20 neurons followed with a ReLU activation function.

We trained two NBFs that (a) terminated after 3 epochs and (b) complete training. As shown in Fig. 1a, the NBF with unfinished training has a boundary that intersects the unsafe region, hence, this barrier function is used to test whether our approach can correctly identify that an NBF does not ensure safety. The fully-trained NBF has a boundary shown in Fig. 1b that separates the safe and unsafe regions.

We next verify the NBFs with our proposed method. NBF (a) fails in barrier function verification and NBF (b) passed all components of the verification after 17.65 seconds(s) in total. The enumerate set procedure takes 0.45s to find 13 activated set that contains the boundary of barrier function. The barrier function verification and the single activated set activation take 5.61s and 5.78s, respectively. The boundary verification finds and verifies 12 intersections of boundary of barrier function and boundary of activated sets within 5.81s. We compared our total run time with state-of-the-art SMT-based verification, isat3. Verification of a 10 neuron NBF with bent-ReLU activation function was reported as 20.84s in [33].

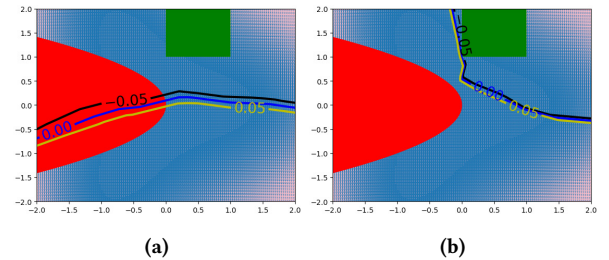


Figure 1: Visualization of neural barrier functions. (a) Barrier function training with insufficient data results in a boundary (black, blue, and yellow lines) that intersects with the unsafe region (red area). Our verification algorithm identifies this safety violation. (b) Training with sufficient data results in a boundary that avoids the unsafe region.

6.2 Obstacle Avoidance

We next evaluate that our proposed method on a controlled system [8]. We consider an Unmanned aerial vehicle (UAVs) flying

Case Study	$n(\mathcal{S})$	$n(\mathcal{V})$	t_e (s)	t_b (s)	t_V (s)	t_S (s)	Total (s)
Darboux (N=20)	13	12	0.45 (2.57%)	5.61 (31.76%)	5.81 (32.91%)	5.78 (32.75%)	17.65
Obs NBF (N=50)	149	186	138.40 (77.75%)	8.31 (4.67%)	22.49 (12.63%)	8.80 (4.94%)	177.99
Obs NCBF (N=50)	149	186	147.78 (75.23%)	8.97 (4.57%)	34.91 (17.77%)	4.78 (2.44%)	196.45
Obs NBF (N=128)	348	429	4302.95 (96.63%)	14.58 (0.33%)	119.88 (2.69%)	15.58 (0.35%)	4452.98
Obs NCBF (N=128)	348	429	4459.18 (96.15%)	14.75 (0.32%)	157.73 (3.40%)	5.88 (0.13%)	4637.54

Table 1: Run-time and ratio of each function to total time over case studies. The proposed algorithm enumerates and verifies $n(\mathcal{S})$ activated sets and $n(\mathcal{V})$ intersections. As the number of neurons increases, the time required to enumerate the sets of activated neurons consumes the largest CPU time.

through a forest avoiding collision with tree trunks. We model the system as a Dubins-style [15] aircraft model. The system state consists of 2-D position and aircraft yaw rate $x := [x_1, x_2, \psi]^T$. We let u denote the control input to manipulate yaw rate and define the dynamics as

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} v \sin \psi \\ v \cos \psi \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ u \end{bmatrix}. \quad (23)$$

We define the state space, initial region and unsafe region as $\mathcal{X}$, $\mathcal{X}_I$ and $\mathcal{X}_U$, respectively as

$$\begin{aligned} \mathcal{X} &: \{x \in \mathbb{R}^3 : x_1, x_2 \in [-2, 2] \times [-2, 2], -\pi/2 < \psi < \pi/2\} \\ \mathcal{X}_I &: \{x \in \mathbb{R}^3 : -0.1 \leq x_1 \leq 0.1, -2 \leq x_2 \leq -1.8, \\ &\quad -\pi/6 < \psi < \pi/6\} \\ \mathcal{X}_U &: \{x \in \mathbb{R}^3 : x_1^2 + x_2^2 \leq 0.04\} \end{aligned} \quad (24)$$

We construct NCBF $b = -b_0(x)$ with one hidden layer consist of 50 neurons. The NCBF b passes the verification within 196.45 seconds(s). The enumerate set procedure takes 147.78s to find 149 activated set that contains the boundary of barrier function. The barrier function verification and the single activated set activation take 8.97s and 4.78s, respectively. The boundary verification finds and verifies 186 intersections of boundary of barrier function and boundary of activated sets within 34.91s.

We evaluated the scalability of our approach by testing on a larger neural network. We construct a NCBF with one hidden layer consist of 128 neurons. The NCBF passes the verification within 4637.54 seconds(s). The enumerate set procedure takes 4459.18s to find 348 activated set that contains the boundary of barrier function. The barrier function verification and the single activated set activation take 14.75s and 5.88s, respectively. The boundary verification finds and verifies 429 intersections of boundary of barrier function and boundary of activated sets within 157.73s.

We next evaluated the ability of our approach to verify safety of a given control law. We assumed v to be 1 and the control law u is designed as:

$$u = -\sin \psi + 3 \cdot \frac{x_1 \cdot \sin \psi + x_2 \cdot \cos \psi}{0.5 + x_1^2 + x_2^2}. \quad (25)$$

We construct the NBF $b := -b_0(x)$ with one hidden layer consist of 50 neurons followed with a ReLU activation function. We then trained an NBF with one hidden layer consisting of 128 neurons followed with a ReLU activation function. The NBF passes the verification within 4452.98 seconds(s).

6.3 Discussion of Case Study Results

Our neural barrier function verification approach relies on decomposing the barrier function b into piecewise linear components. Since enumerating all possible piecewise-linear regions is worst-case exponential in the number of neurons, we attempt to mitigate the complexity by only verifying the barrier function at the safety boundary. Our numerical results suggest that this significantly improves the tractability of the verification, since there are only 149 activation sets that must be verified for a neural network of 50 nodes, and only 348 sets that must be verified for a neural network of 128 nodes.

The trade-off of this approach is that it requires an initial stage of enumerating all activation regions that comprise the boundary of the safe set. Indeed, this step requires the bulk of the computation time (75% of the computation time for the 50-node neural barrier function and 96% of the computation time for the 128-node neural barrier function). Reducing this computation time, especially for higher-dimensional systems, represents a direction for future work. In particular, we note that in our approach we did not attempt to exploit problem sparsity or optimize the code for enumerating and verifying the boundary regions.

7 CONCLUSIONS

This paper studied the problem of verifying safety of control barrier functions that are represented as neural networks. We formulated safety conditions based on two observations regarding neural barrier functions. First, it is sufficient to verify safety by examining a subset of safety-critical states at the boundary of the safe region. Second, the piecewise-linearity of the widely-used ReLU activation function enables decomposition of the verification problem into a set of nonlinear programs, which are equivalent to SOS programs when the system dynamics are polynomial. Based on these observations, we proposed a neural barrier certificate verification algorithm that characterizes the boundary of the safe region as the union of activation sets of the neurons, and then verifies each activation set. We evaluated our approach on two nonlinear systems and demonstrated its scalability to neural networks of up to 128 neurons.

REFERENCES

- [1] Alessandro Abate, Daniele Ahmed, Alec Edwards, Mirco Giacobbe, and Andrea Peruffo. 2021. FOSSIL: a software tool for the formal synthesis of lyapunov functions and barrier certificates using neural networks. In *Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control*. 1–11.
- [2] Devansh R Agrawal and Dimitra Panagou. 2021. Safe control synthesis via input constrained control barrier functions. In *2021 60th IEEE Conference on Decision*

- and Control (CDC). IEEE, 6113–6118.
- [3] Amir Ali Ahmadi, Georgina Hall, Antonis Papachristodoulou, James Saunderson, and Yang Zheng. 2017. Improving efficiency and scalability of sum of squares optimization: Recent advances and limitations. In *2017 IEEE 56th annual conference on decision and control (CDC)*. IEEE, 453–462.
 - [4] Amir Ali Ahmadi and Anirudha Majumdar. 2016. Some applications of polynomial optimization in operations research and real-time decision making. *Optimization Letters* 10, 4 (2016), 709–729.
 - [5] Aaron D Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. 2019. Control barrier functions: Theory and applications. In *2019 18th European control conference (ECC)*. IEEE, 3420–3431.
 - [6] Aaron D Ames, Jessy W Grizzle, and Paulo Tabuada. 2014. Control barrier function based quadratic programs with application to adaptive cruise control. In *53rd IEEE Conference on Decision and Control*. IEEE, 6271–6278.
 - [7] Jean-Pierre Aubin, Alexandre M Bayen, and Patrick Saint-Pierre. 2011. *Viability Theory: New Directions*. Springer Science & Business Media.
 - [8] Andrew J Barry, Anirudha Majumdar, and Russ Tedrake. 2012. Safety verification of reactive controllers for UAV flight in cluttered environments using barrier certificates. In *2012 IEEE International Conference on Robotics and Automation*. IEEE, 484–490.
 - [9] Felix Berkenkamp, Matteo Turchetta, Angela Schoellig, and Andreas Krause. 2017. Safe model-based reinforcement learning with stability guarantees. *Advances in neural information processing systems* 30 (2017).
 - [10] Franco Blanchini and Stefano Miani. 2008. *Set-theoretic methods in control*. Vol. 78. Springer.
 - [11] Nicholas Boffi, Stephen Tu, Nikolai Matni, Jean-Jacques Slotine, and Vikas Sindhwani. 2021. Learning stability certificates from data. In *Conference on Robot Learning*. PMLR, 1341–1350.
 - [12] Andrew Clark. 2021. Verification and synthesis of control barrier functions. In *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 6105–6112.
 - [13] Charles Dawson, Sicun Gao, and Chuchu Fan. 2022. Safe Control with Learned Certificates: A Survey of Neural Lyapunov, Barrier, and Contraction methods. *arXiv preprint arXiv:2202.11762* (2022).
 - [14] Charles Dawson, Zengyi Qin, Sicun Gao, and Chuchu Fan. 2022. Safe nonlinear control using robust neural Lyapunov-barrier functions. In *Conference on Robot Learning*. PMLR, 1724–1735.
 - [15] Lester E Dubins. 1957. On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents. *American Journal of mathematics* 79, 3 (1957), 497–516.
 - [16] Shao-Chen Hsu, Xiangru Xu, and Aaron D Ames. 2015. Control barrier function based quadratic programs with application to bipedal robotic walking. In *2015 American Control Conference (ACC)*. IEEE, 4542–4548.
 - [17] Guy Katz, Clark Barrett, David L Dill, Kyle Julian, and Mykel J Kochenderfer. 2017. Reluplex: An efficient SMT solver for verifying deep neural networks. In *International conference on computer aided verification*. Springer, 97–117.
 - [18] Guy Katz, Derek A Huang, Duligur Ibeling, Kyle Julian, Christopher Lazarus, Rachel Lim, Parth Shah, Shantanu Thakoor, Haoze Wu, Aleksandar Zeljic, et al. 2019. The marabou framework for verification and analysis of deep neural networks. In *International Conference on Computer Aided Verification*. Springer, 443–452.
 - [19] Haitong Ma, Changliu Liu, Shengbo Eben Li, Sifa Zheng, and Jianyu Chen. 2021. Joint Synthesis of Safety Certificate and Safe Control Policy using Constrained Reinforcement Learning. *arXiv preprint arXiv:2111.07695* (2021).
 - [20] Jiří Matoušek and Bernd Gärtner. 2007. *Understanding and using linear programming*. Vol. 33. Springer.
 - [21] Matthew Newton and Antonis Papachristodoulou. 2021. Neural network verification using polynomial optimisation. In *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 5092–5097.
 - [22] Pablo A Parrilo. 2003. Semidefinite programming relaxations for semialgebraic problems. *Mathematical programming* 96, 2 (2003), 293–320.
 - [23] Stephen Prajna, Ali Jadbabaie, and George J Pappas. 2007. A framework for worst-case and stochastic safety verification using barrier certificates. *IEEE Trans. Automat. Control* 52, 8 (2007), 1415–1428.
 - [24] Zengyi Qin, Kaiqing Zhang, Yuxiao Chen, Jingkai Chen, and Chuchu Fan. 2021. Learning safe multi-agent control with decentralized neural barrier certificates. *arXiv preprint arXiv:2101.05436* (2021).
 - [25] Karsten Scheibler, Stefan Kupferschmid, and Bernd Becker. 2013. Recent Improvements in the SMT Solver iSAT. *MBMV 13* (2013), 231–241.
 - [26] Meng Sha, Xin Chen, Yuzhe Ji, Qingye Zhao, Zhengfeng Yang, Wang Lin, Enyi Tang, Qiguang Chen, and Xuandong Li. 2021. Synthesizing barrier certificates of neural network controlled continuous systems via approximations. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 631–636.
 - [27] Andrew Taylor, Andrew Singletary, Yisong Yue, and Aaron Ames. 2020. Learning for safety-critical control with control barrier functions. In *Learning for Dynamics and Control*. PMLR, 708–717.
 - [28] Tianhao Wei and Changliu Liu. 2019. Safe control algorithms using energy functions: A unified framework, benchmark, and new directions. In *2019 IEEE 58th Conference on Decision and Control (CDC)*. IEEE, 238–243.
 - [29] Tianhao Wei and Changliu Liu. 2022. Safe Control with Neural Network Dynamic Models. In *Learning for Dynamics and Control Conference*. PMLR, 739–750.
 - [30] Weiming Xiang, Hoang-Dung Tran, and Taylor T Johnson. 2018. Output reachable set estimation and verification for multilayer neural networks. *IEEE transactions on neural networks and learning systems* 29, 11 (2018), 5777–5783.
 - [31] Xiangru Xu, Jessy W Grizzle, Paulo Tabuada, and Aaron D Ames. 2017. Correctness guarantees for the composition of lane keeping and adaptive cruise control. *IEEE Transactions on Automation Science and Engineering* 15, 3 (2017), 1216–1229.
 - [32] Xia Zeng, Wang Lin, Zhengfeng Yang, Xin Chen, and Lilei Wang. 2016. Darboux-type barrier certificates for safety verification of nonlinear hybrid systems. In *Proceedings of the 13th International Conference on Embedded Software*. 1–10.
 - [33] Hengjun Zhao, Xia Zeng, Taolue Chen, and Zhiming Liu. 2020. Synthesizing barrier certificates using neural networks. In *Proceedings of the 23rd International Conference on Hybrid Systems: Computation and Control*. 1–11.
 - [34] Hengjun Zhao, Xia Zeng, Taolue Chen, Zhiming Liu, and Jim Woodcock. 2021. Learning safe neural network controllers with barrier certificates. *Formal Aspects of Computing* 33, 3 (2021), 437–455.
 - [35] Qingye Zhao, Xin Chen, Zhuoyu Zhao, Yifan Zhang, Enyi Tang, and Xuandong Li. 2022. Verifying Neural Network Controlled Systems Using Neural Networks. In *25th ACM International Conference on Hybrid Systems: Computation and Control*. 1–11.
 - [36] Weiye Zhao, Tairan He, and Changliu Liu. 2021. Model-free safe control for zero-violation reinforcement learning. In *5th Annual Conference on Robot Learning*.
 - [37] Weiye Zhao, Tairan He, Tianhao Wei, Simin Liu, and Changliu Liu. 2022. Safety Index Synthesis via Sum-of-Squares Programming. *arXiv preprint arXiv:2209.09134* (2022).

APPENDIX

This section contains proofs of Lemma 4 and Proposition 1.

PROOF OF LEMMA 4. Let γ be a path satisfying the conditions of the lemma. If $\gamma(t') \in U_i \cap C$, then condition (i) holds with $t'' = t$. If $\gamma(t') \in U_i \setminus C$, then $\gamma(t') \in \Phi_i$. Since $\gamma(0) \notin \Phi_i$ and γ is continuous, there exists t''' such that $\gamma(t''') \in \partial\Phi_i = \partial\Psi_i = D_i$. Hence (ii) holds.

Next, we prove that D_i is a connected component of ∂C_0 . We first show that D_i is contained in ∂C_0 , and then show that D_i is connected. Finally, we show that D_i is a connected component of ∂C_0 .

Since Φ_i is a connected component of $\mathbb{R}^n \setminus C_0$, we have $D_i = \partial\Psi_i = \partial\Phi_i \subseteq \partial C_0$.

In order to prove that D_i is connected, we first show that Ψ_i is connected. Let $x_1, x_2 \in \Psi_i$. If x_1, x_2 are both in C_0 or in the same connected component of $\mathbb{R}^n \setminus C_0$, then they are connected. If $x_1 \in C_0$ and $x_2 \notin C_0$, we note that the connected component of $\mathbb{R}^n \setminus C_0$ must share a boundary with C_0 ; otherwise, it would not be a maximal connected set in the complement of C_0 . Hence we can find a path from x_2 to the boundary of C_0 , and then exploit connectedness of C_0 to find a path to x_1 . Finally, suppose that x_1 and x_2 are in different connected components of $\mathbb{R}^n \setminus C_0$. Let γ be a path from x_1 and x_2 . If γ is not contained in Ψ_i , then γ must reach the boundary D_i . Let x'_1 and x'_2 denote the first and last points on γ that lie on D_i , respectively. Then we can exploit connectedness of C_0 to construct a path inside Ψ_i by replacing the segment of γ from x'_1 to x'_2 with a path contained in C_0 .

Now, we have that Φ_i and Ψ_i are connected and $\Phi_i \cup \Psi_i = \mathbb{R}^n$. Suppose that D_i is not connected. Let F_1 and F_2 denote two disjoint sets that do not share a limit point and $F_1 \cup F_2 = D_i$. Let $x_1 \in \Phi_i$ and $x_2 \in \Psi_i$. Let γ_1 and γ_2 denote paths from x_1 to x_2 that traverse only F_1 and F_2 , respectively. Since $\mathbb{R}^n$ is simply connected, we can construct a homotopy between these two paths. By continuity, there exists a path within the homotopy such that the path does not intersect F_1 or F_2 , and hence does not intersect D_i , a contradiction.

Finally, suppose that $x \in \partial C_0$, x is connected to D_i , and yet $x \notin D_i$. By definition, x lies at the boundary of C_0 and one of the other connected components of $\mathbb{R}^n \setminus C_0$. This, however, implies that these connected components are connected to each other via the path contained in D_i , a contradiction. Hence D_i is a connected component of ∂C . $\square$

PROOF OF PROPOSITION 1. As a preliminary, we show that when $x \in D_i$, $\mathcal{T}_{\Psi_i}(x) = \mathcal{T}_C(x)$. Indeed, by Lemma 4, D_i is a connected component of the boundary of C_0 and hence a connected component of ∂C_0 . Hence $\text{dist}(x + \tau z, C) = \text{dist}(x + \tau z, \Psi_i)$ for τ sufficiently small, implying that $z \in \mathcal{T}_C(x)$ iff $z \in \mathcal{T}_{\Psi_i}(x)$. It therefore suffices to compute $\mathcal{T}_C(x)$ for $x \in D_i$.

Define $\bar{X}_0(S) = \bar{X}(S) \cap C$. We will first show that, for all x with $b(x) = 0$,

$$\mathcal{T}_C(x) = \bigcup_{S \in S(x)} \mathcal{T}_{\bar{X}_0(S)}(x). \quad (26)$$

We observe that

$$\text{dist}\left(x, C \setminus \bigcup_{S \in S(x)} \bar{X}_0(S)\right) > 0,$$

and hence

$$\text{dist}(x + \tau z, C) = \min_{S \in S(x)} \text{dist}(x + \tau z, \bar{X}_0(S))$$

for τ sufficiently small.

Suppose that $z \in \mathcal{T}_{\bar{X}_0(S)}(x)$. Then for any $\tau \geq 0$, $\text{dist}(x + \tau z, C) \leq \text{dist}(x + \tau z, \bar{X}_0(S))$, and hence

$$\liminf_{\tau \rightarrow 0} \frac{\text{dist}(x + \tau z, C)}{\tau} \leq \liminf_{\tau \rightarrow 0} \frac{\text{dist}(x + \tau z, \bar{X}_0(S))}{\tau} = 0.$$

We therefore have $z \in \mathcal{T}_C(x)$.

Now, suppose that $z \in \mathcal{T}_C(x)$ and yet $z \notin \bigcup_{S \in S(x)} \mathcal{T}_{\bar{X}_0(S)}(x)$. Then for all $S \in S(x)$, there exists $\epsilon_S > 0$ such that

$$\liminf_{\tau \rightarrow 0} \frac{\text{dist}(x + \tau z, \bar{X}_0(S))}{\tau} = \epsilon_S.$$

Let $\bar{\epsilon} = \min \{\epsilon_S : S \in S(x)\}$. For any $\delta \in (0, \bar{\epsilon})$, there exists $\bar{\tau} > 0$ such that $\tau < \bar{\tau}$ implies

$$\begin{aligned} \frac{\text{dist}(x + \tau z, C)}{\tau} &= \min_{S \in S(x)} \frac{\text{dist}(x + \tau z, \bar{X}_0(S))}{\tau} \\ &> \delta \end{aligned}$$

implying that $\liminf_{\tau \rightarrow 0} \frac{\text{dist}(x + \tau z, C)}{\tau} > 0$ and hence $z \notin \mathcal{T}_C(x)$. This contradiction implies (26).

It now suffices to show that, for each $S \in S(x)$,

$$\mathcal{T}_{\bar{X}_0(S)}(x) = \left(\bigcap_{(i,j) \in T(x)} \{z : \bar{W}_{ij}^T(S)^T z \geq 0\} \right) \cap \{z : \bar{W}(S)^T z \geq 0\}.$$

We have that each $\bar{X}_0(S)$ is given by

$$\begin{aligned} \bar{X}_0(S) &= \{x' : \bar{W}_{ij}(S)^T x' + r_{ij}(S) \geq 0 \forall (i, j) \in S\} \\ &\cap \{x' : \bar{W}_{ij}(S)^T x' + r_{ij}(S) \leq 0 \forall (i, j) \notin S\} \cap \{x' : \bar{W}(S)^T x' + r(S) \geq 0\}, \end{aligned}$$

thus matching the conditions of Lemma 3 when each g_k function is affine. Furthermore, the set $J(x)$ is equal to the set of functions that are exactly zero at x , which consists of $\{\bar{W}_{ij}(S)^T x + \bar{r}_{ij}(S) : (i, j) \in$

$T(x)\}$ together with $\bar{W}(S)^T x + \bar{r}(S)$. This observation combined with Lemma 3 gives the desired result. $\square$

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009